

[< Go to the original](#)

Model Context Protocol (MCP): An End-To-End Tutorial With Hands-On Project

What is MCP? How to create an MCP Server that brings news from a web site with Claude Desktop?



Follow



Data Science Collective a11y-light ~8 min read · March 18, 2025 (Updated: March 19, 2025) ·

Free: Yes

The world of artificial intelligence is constantly evolving and we wake up to new news almost every day. What we need to learn now is **MCP (Model Context Protocol)**. Before moving on to what it is and its purpose, let's look at what the protocol means.

What is Protocol?

In the computer world, a protocol is a set of rules that determine how two systems will communicate with each other. **Protocols** regulate data transfer in computer networks, internet communication, and between software systems.

For example:

- **HTTP (Hypertext Transfer Protocol):** Allows websites to communicate with browsers.
- **TCP/IP (Transmission Control Protocol/Internet Protocol):** Defines how data packets on the internet will be routed.

Now we can move on to MCP and its features.

What is Model Context Protocol (MCP)?

The Model Context Protocol (MCP) is an open protocol that enables large language models (LLMs) to integrate with external data sources and tools in a standardized way. Developed by Anthropic, this protocol makes it easy for AI models to work seamlessly with a variety of tools and data sources.

MCP can be likened to the USB-C port, which has become a global standard for device connections. Just as USB-C provides a common connection point between different devices, MCP enables AI systems to communicate with data and tools in a standard way.

Why Use MCP?

MCP functions similarly to APIs, but has a wider potential for use. While traditional APIs require a separate implementation for each integration, a single integration with MCP provides access to many different data sources and tools.

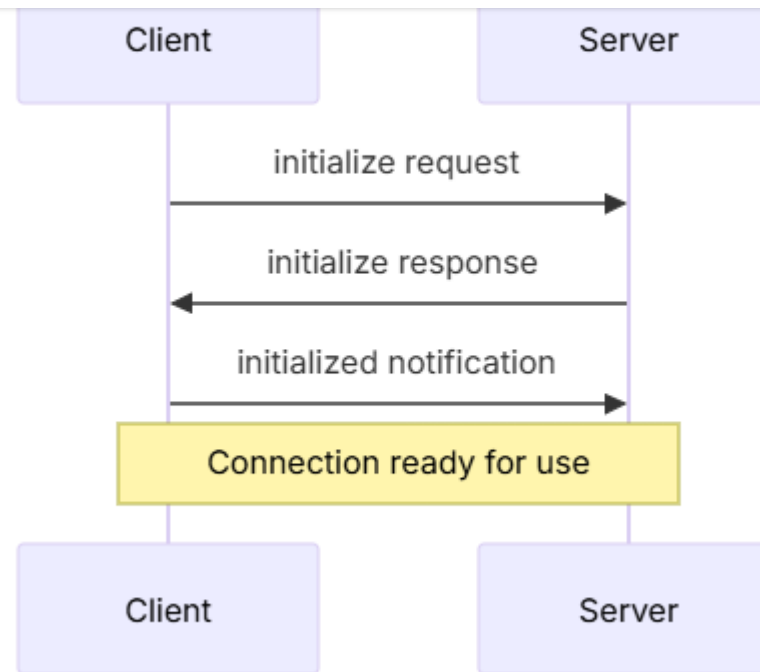
Architecture of MCP

MCP is based on a simple client-server architecture. An application can connect to multiple MCP servers at the same time. The structure consists of the following components:

- **MCP Hosts:** Applications act as MCP hosts to access data or tools.
- **MCP Clients:** Clients within the host establish one-to-one connections with MCP servers.
- **MCP Servers:** Lightweight, provide specific functionality through MCP, and can connect to local or remote data sources.
- **Local Data Sources:** Data that can be accessed by MCP servers, such as files and databases.
- **Remote Services:** External Internet-based APIs that MCP servers can access.

Connection Lifecycle

1. Initialization



<https://modelcontextprotocol.io/docs/concepts/architecture>

- The client sends an initialize request to the server, containing its own protocol version and capabilities.
- The server responds with its own protocol version and capabilities.
- The client sends the initialized notification.
- The connection is established and the message exchange begins.

2. Message Exchange

transmitted.

3. Termination

- The client or server can terminate the connection.

Key Features of MCP

MCP uses the JSON-RPC 2.0 message format to communicate between the client and server. Some of the protocol's prominent features are:

- **Resources:** Data and content presented to the user or AI model.
- **Prompts:** Predefined messages and workflows prepared for users.
- **Tools:** Functions that the AI model can run.

Hands On Project

In this project, we will create a structure that brings the latest news from a website.

command below. You can download it to Windows with the second command.

Copy

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | :
```

```
Last login: Tue Feb 25 22:31:48 on ttys009
[busekoseoglu@Buse-Air ~ % curl -LsSf https://astral.sh/uv/install.sh | sh
downloading uv 0.6.7 aarch64-apple-darwin
no checksums to verify
installing to /Users/busekoseoglu/.local/bin
  uv
  uvx
everything's installed!

To add $HOME/.local/bin to your PATH, either restart your shell or run:

    source $HOME/.local/bin/env (sh, bash, zsh)
    source $HOME/.local/bin/env.fish (fish)
WARNING: The following commands are shadowed by other commands in your PATH: uv uvx
[busekoseoglu@Buse-Air ~ % source $HOME/.local/bin/env
```

Don't forget to restart your terminal after doing this.

Then we will create a directory for our project. To do this, open the terminal in the directory where you want to create the project and run the following

- The first command creates a project file in the directory you are in.
- The second command allows you to enter this file directory.

Copy

```
uv init mcp-server-project
```

```
cd mcp-server-project
```

```
busekoseoglu@Buse-Air LLM Portfolio % uv init mcp-server-project
Initialized project `mcp-server-project` at `/Users/busekoseoglu/Desktop/PROJELER/LLM Portfolio/mcp-server-project`
busekoseoglu@Buse-Air LLM Portfolio % cd mcp-server-project
busekoseoglu@Buse-Air mcp-server-project %
```

Then we will create a virtual environment and install our packages. For this we use the uv package manager.

Copy

```
# create virtual env
uv venv
```

```
# activate for macos/linux
source .venv/bin/activate
```

```
# activate for windows
```

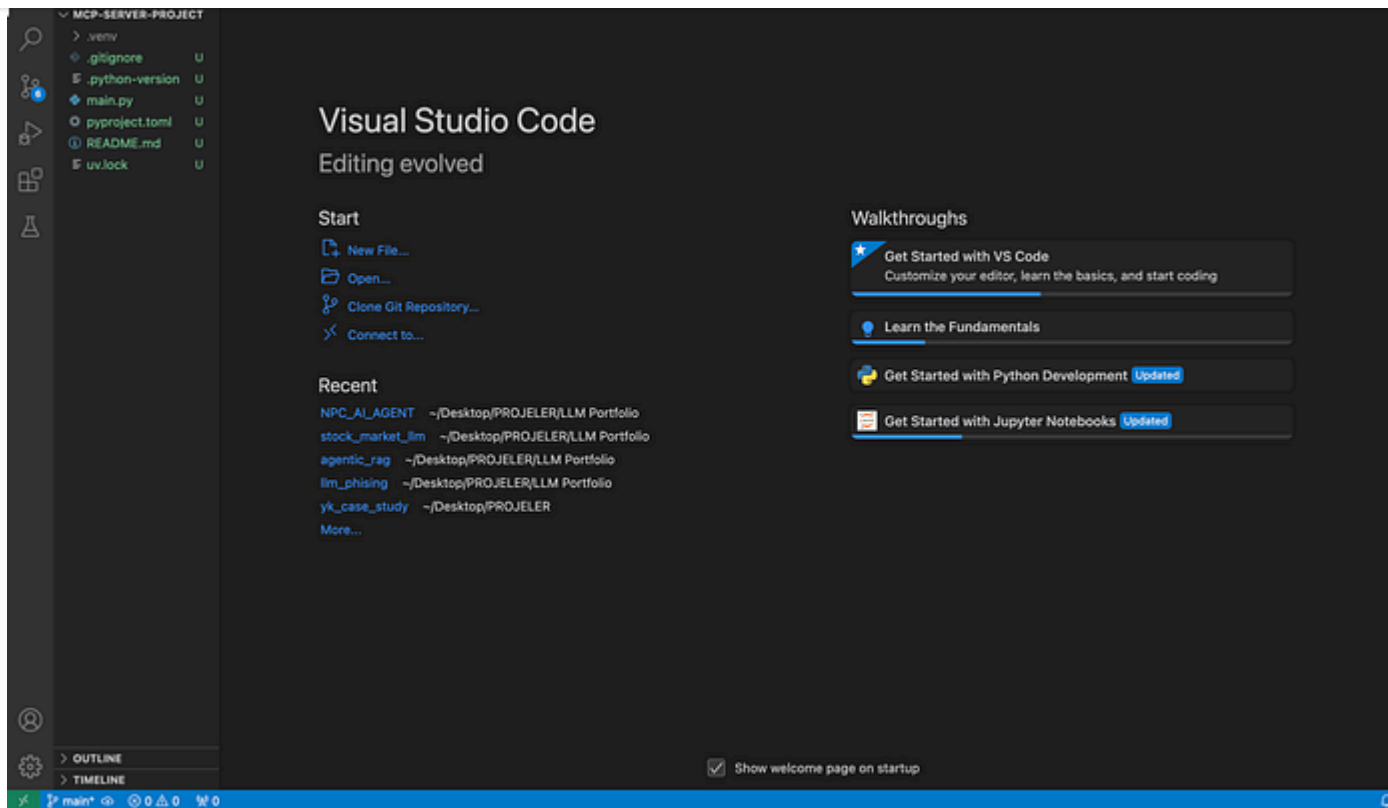


```
# Install dependencies
```

```
uv add "mcp[cli]" httpx bs4 dotenv
```

```
[busekoseoglu@Buse-Air mcp-server-project % uv venv
Using CPython 3.12.8 interpreter at: /opt/homebrew/opt/python@3.12/bin/python3.12
Creating virtual environment at: .venv
Activate with: source .venv/bin/activate
[busekoseoglu@Buse-Air mcp-server-project % source .venv/bin/activate
(mcp-server-project) busekoseoglu@Buse-Air mcp-server-project % uv add "mcp[cli]" httpx
Resolved 27 packages in 1.46s
Prepared 25 packages in 1.14s
Installed 25 packages in 31ms
+ annotated-types==0.7.0
+ anyio==4.9.0
+ certifi==2025.1.31
+ click==8.1.8
+ h11==0.14.0
+ httpcore==1.0.7
+ httpx==0.28.1
```

Now we need to open the folder we created in vscode. When we open it, we see that our file structure is ready on the left.



Here we will update the **main.py** file.

Copy

```
from mcp.server.fastmcp import FastMCP
from dotenv import load_dotenv
import httpx
import os
from bs4 import BeautifulSoup
```

```
load_dotenv()

# initialize server
mcp = FastMCP("tech_news")

USER_AGENT = "news-app/1.0"

NEWS_SITES = {
    "arstechnica": "https://arstechnica.com"
}

async def fetch_news(url: str):
    """It pulls and summarizes the latest news from the specified news site."""
    async with httpx.AsyncClient() as client:
        try:
            response = await client.get(url, timeout=30.0)
            soup = BeautifulSoup(response.text, "html.parser")
            paragraphs = soup.find_all("p")
            text = " ".join([p.get_text() for p in paragraphs[:5]])
            return text
        except httpx.TimeoutException:
            return "Timeout error"

@mcp.tool()
async def get_tech_news(source: str):
    """
    Fetches the latest news from a specific tech news source.

    Args:
```

```
"""
A brief summary of the latest news.
"""

if source not in NEWS_SITES:
    raise ValueError(f"Source {source} is not supported.")

news_text = await fetch_news(NEWS_SITES[source])
return news_text

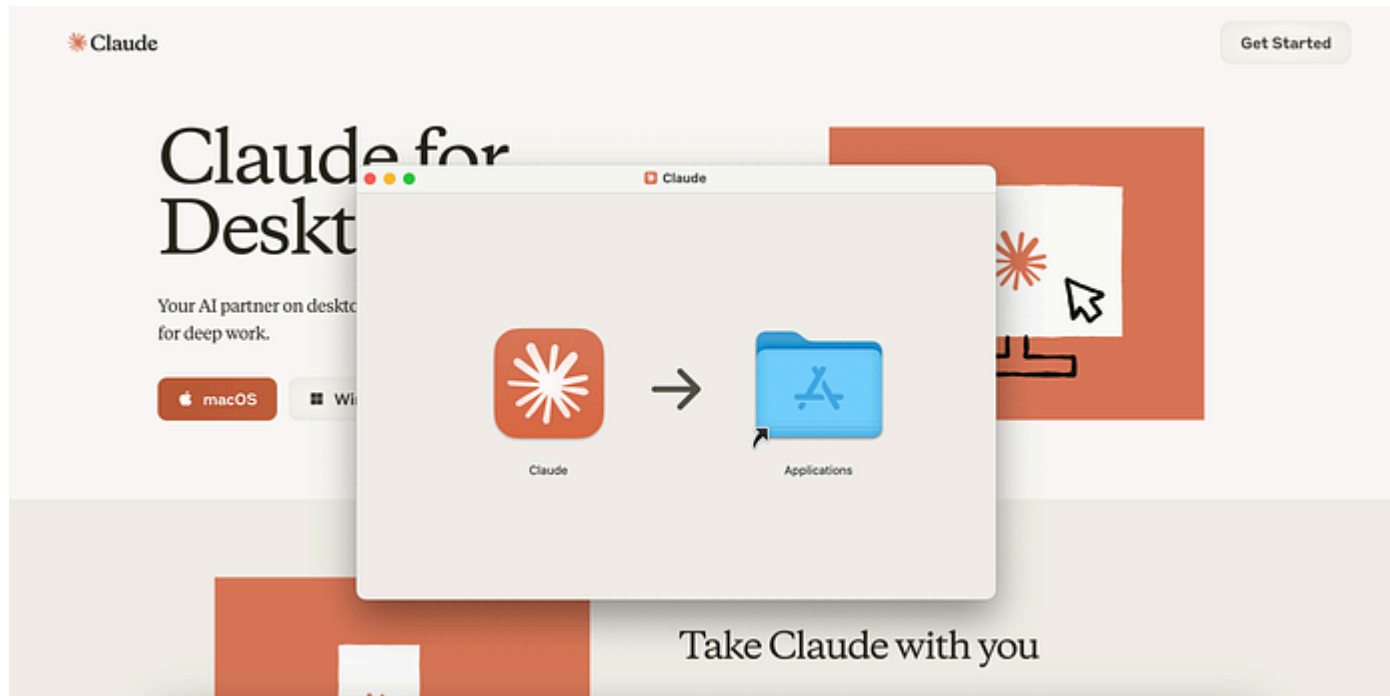
if __name__ == "__main__":
    mcp.run(transport="stdio")
```

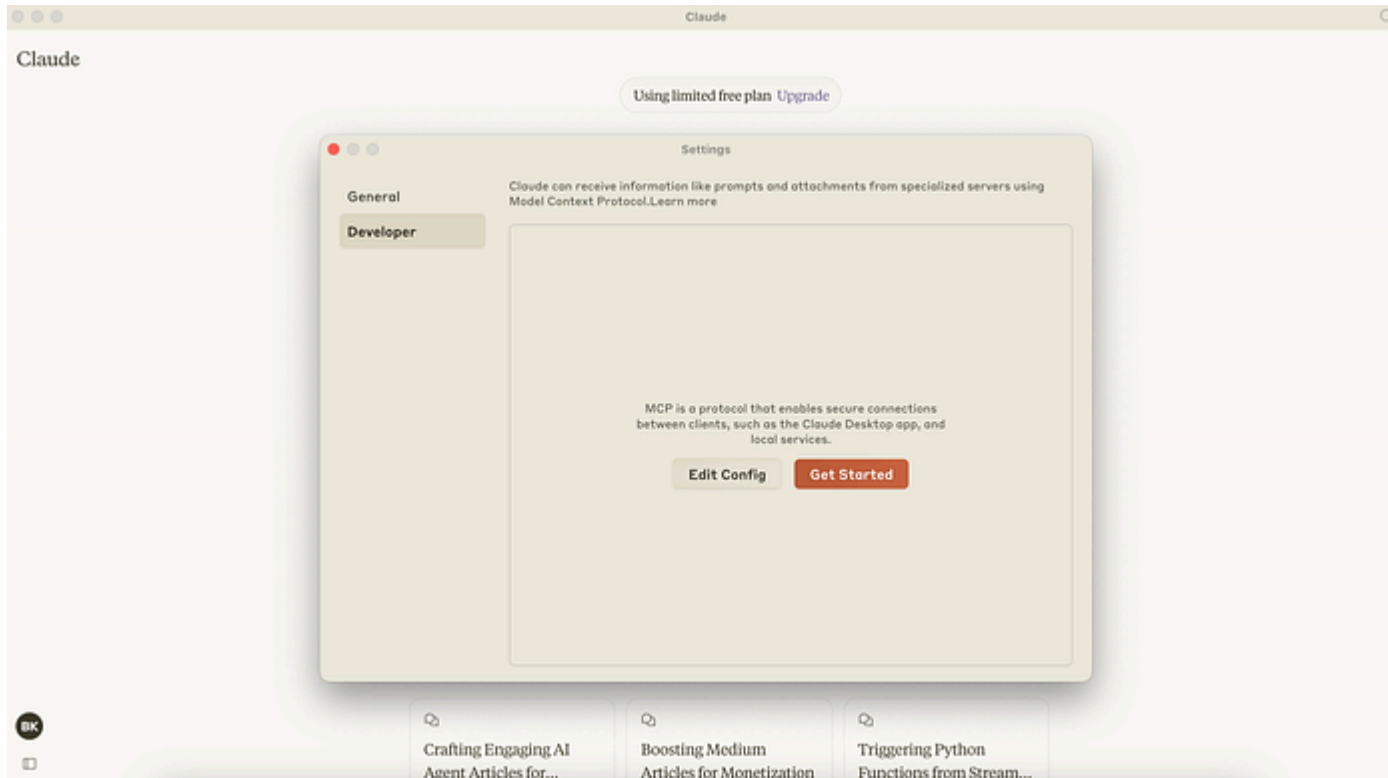
The code above allows the latest news to be retrieved from the given site. It works step by step as follows.

1. First, we make the necessary imports.
2. No API was used in this code, but if you use the API, you can use `load_dotenv()` to get your keys.
3. We initialize our server with **FastMCP**.
4. **"news-app/1.0"** is the application name we gave.
5. **NEWS_SITES** contains the sites from which the news will be retrieved. You can add more sites here if you want.

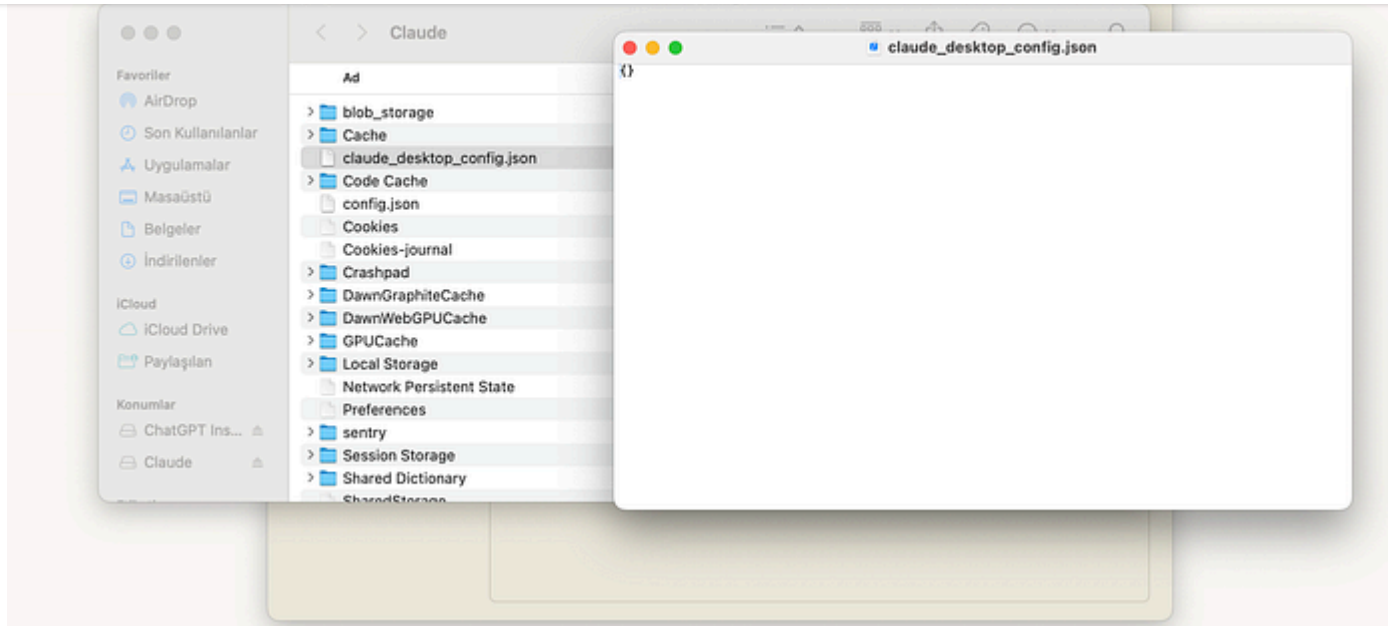
7. The `get_tech_news()` function is our tool here. We specify that this is a tool by adding the `@mcp.tool()` decorator to the function. Having a docstring in this function is important for the model to understand how the tool works.
8. Our MCP Server is running with `mcp.run(transport="stdio")`. But we will not run the server via vscode.

Claude desktop can directly run the MCP Server we prepared, so we will use Claude Desktop. You can download it from here: <https://claude.ai/download>.





When we click on **edit config**, it opens a folder for us. From this folder, we play the **claude_desktop_config.json** file with a text editor. We will enter our server information here.



This file contains how our server will be run by claude. There are a few points to note:

1. **mcp-server-project** is the name of the project file we created. If you created it with a different name, you can change this.
2. You should add the path where the uv package manager is located to the command section. In this section, only "uv" is written in the document, but it did not work that way for me, so I gave the path. You can run the "**which uv**" command in the terminal to find the path.

path.

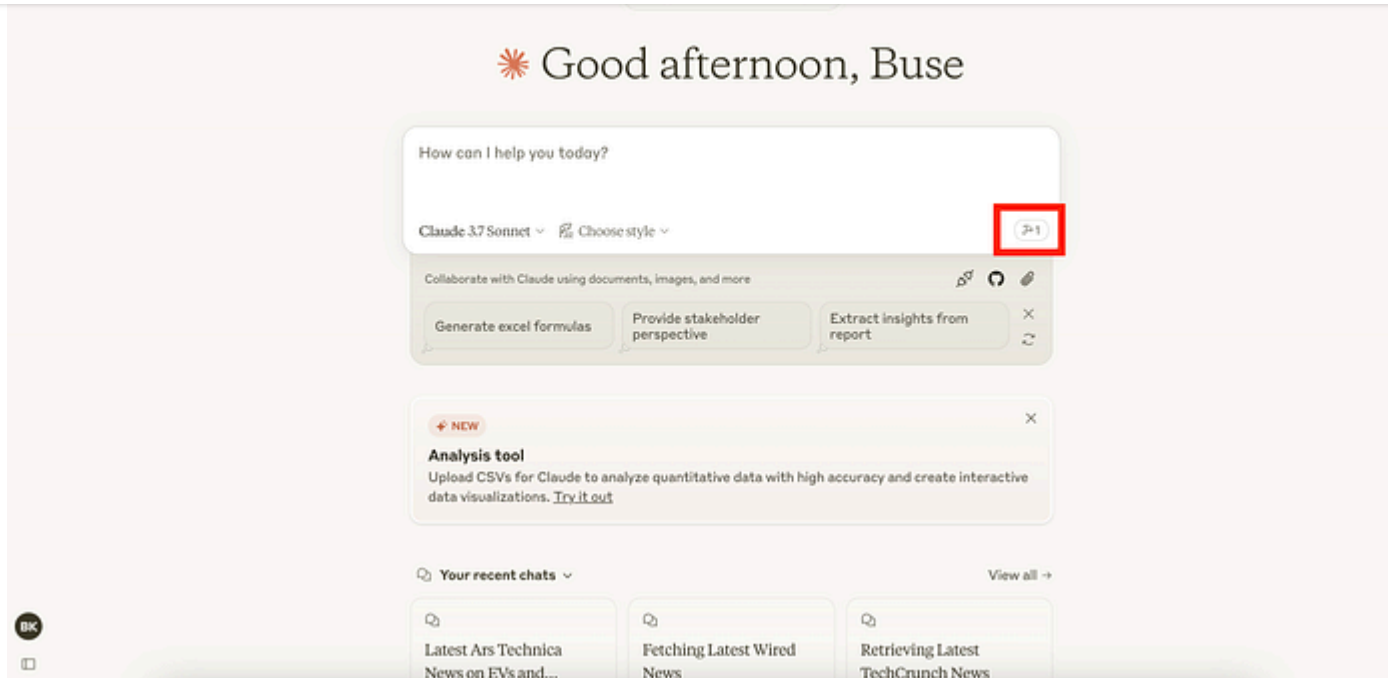
4. You can **save** and close the file.

Copy

```
{
  "mcpServers": {
    "mcp-server-project": {
      "command": "/Users/busekoseoglu/.local/bin/uv",
      "args": [
        "--directory",
        "/Users/busekoseoglu/Desktop/PROJELER/LLM Portfolio/mcp-server-",
        "run",
        "main.py"
      ]
    }
  }
}
```

```
{
  "mcpServers": {
    "mcp-server-project": {
      "command": "/Users/busekoseoglu/.local/bin/uv",
      "args": [
        "--directory",
        "/Users/busekoseoglu/Desktop/PROJELER/LLM Portfolio/mcp-server-project",
        "run",
        "main.py"
      ]
    }
  }
}
```

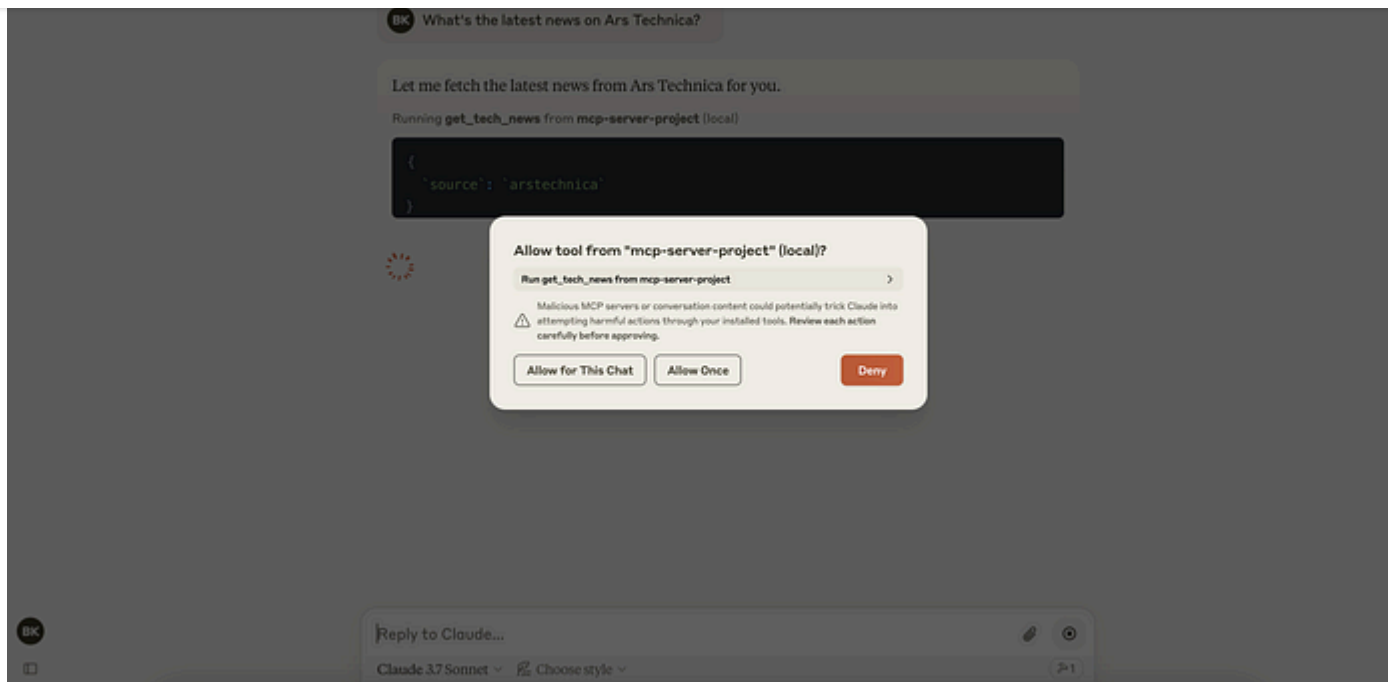
We have completed our server configurations, now we can try it on Claude desktop. When you open Claude desktop, if there is a problem, you can see error pop-ups on the top right. You need to click and examine the log files and solve the error. If you do not receive an error, you should see the hammer in the red square.



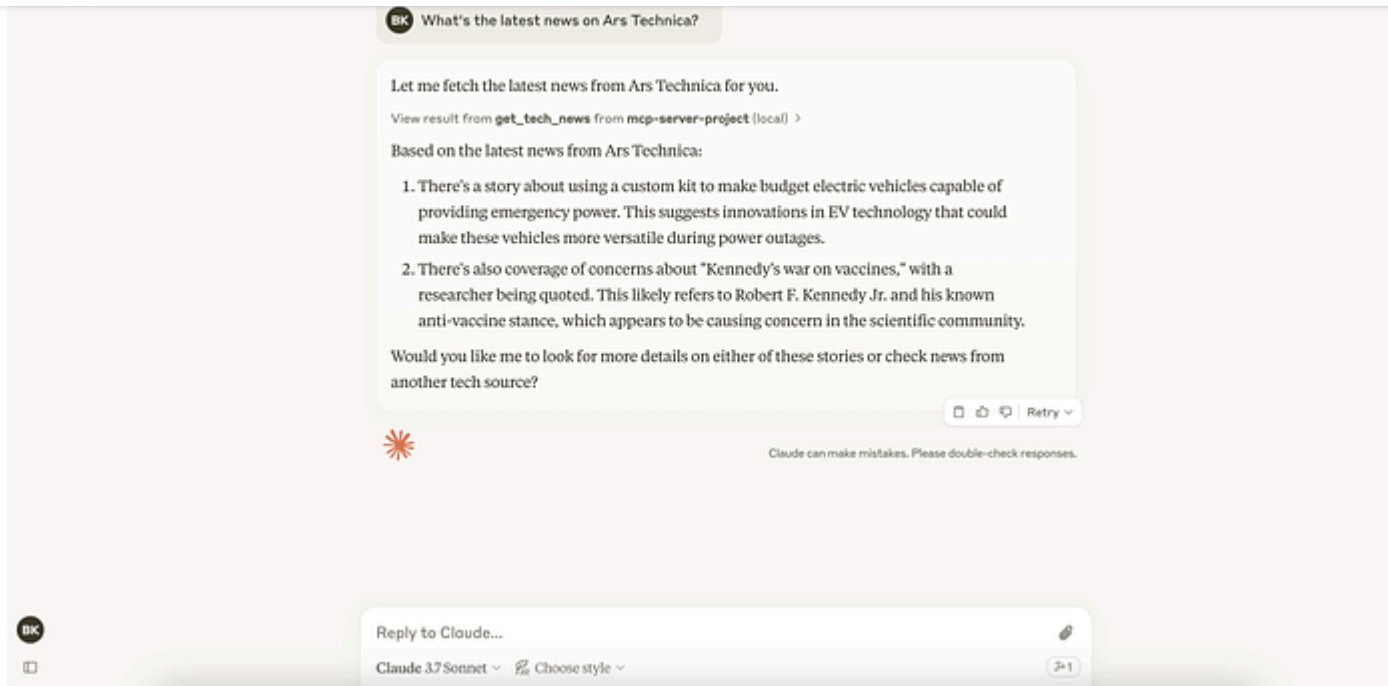
When you click on Hammer, it shows you the available MCP Tools as below.



We came to the test section and I asked the question "What's the latest news on Ars Technica?" On the screen, it asked me for permission to use the tool. Here Claude understands that he needs to use a tool.



After giving permission, it used my tool to retrieve the answers and return the latest news to me.



Congratulations, you have set up a simple MCP Server using Python. I worked with a simple scenario to both learn how it works and to show you. These can of course be expanded.

I hope it was useful. Don't forget to follow me, clap me 50 times and leave a comment. If you want to check out my other articles:

How to Create Agentic RAG with Self Evaluation Mechanism

Building a YouTube Transcript Agent with LangChain and Ollama: A Step-by-Step Guide

Learn how to extract, summarize, and analyze YouTube video transcripts using LangChain, Ollama, and Python.

gopubby.com

Understanding Vector Databases — 1: Basic Concepts and Working Principle

Entering the World of Semantic Search and Similarity-Based Querying

gopenai.com

References:

modelcontextprotocol.io

For Server Developers - Model Context Protocol

Get started building your own server to use in Claude for Desktop and other clients.

modelcontextprotocol.io

Specification (Latest)

 Protocol Revision: 2024-11-05 Model Context Protocol (MCP) is an open protocol that enables seamless integration...

modelcontextprotocol.io

What is Model Context Protocol (MCP)? How it simplifies AI integrations compared to APIs | AI...

[norahsakal.com](#)

GitHub - alejandro-ao/mcp-server-example: A simple MCP server to search for documentation...

A simple MCP server to search for documentation (tutorial) - alejandro-ao/mcp-server-example

[github.com](#)

Introducing the Model Context Protocol

The Model Context Protocol (MCP) is an open standard for connecting AI assistants to the systems where data lives...

[anthropic.com](#)

[#mcps](#) [#mcp-server](#) [#claude](#) [#llm-applications](#) [#model-context-protocol](#)